> **Archivo:** `entrega/documento-entrega.md`
> **Objetivo:** Documento de entrega · Taller A2A (Curso 2 · Sesión 1)
> **Grupo del curso:** Fabio Durán - Gustavo Reyes - Ezequiel Muñoz
> **Creación:** 20261110 1941 · **Última modificación:** 20261111 1224

# Documento de entrega · Taller A2A (Curso 2 · Sesión 1)

> **Cómo leer este documento.** Sigue los pasos del taller (1 a 8) y agrega dos secciones de cierre: cómo correr el proyecto (9) y las mejoras de robustez que implementamos sobre el coordinador (10). El PDF se genera con `uv run tools/exportar_pdf.py`. Las capturas `01-saltos.png` / `03-tunel.png` se generaron desde la corrida real (son reemplazables por capturas de pantalla manuales). Fabio y Gustavo completan sus blueprints (plantillas ya creadas).

### 0. Resumen ejecutivo

Construimos una red de **agentes que hablan por A2A** (Agent-to-Agent) usando **LangGraph nativo**: cada agente se publica con `langgraph dev` y se descubre y se invoca por su **Agent Card**, sin pegamento propietario. Sobre esa base levantamos un **coordinador/supervisor** que, dada una pregunta del usuario, decide a qué agente mandarla leyendo las fichas del catálogo, y reenvía por A2A.

La gracia del trabajo es que funciona **entre dos computadores en redes distintas** (un Mac y un Windows) **solo por túneles HTTP de Cloudflare** —sin LAN, sin abrir puertos, sin nada por TCP— y que el coordinador trata las respuestas remotas como **datos no confiables**. Además endurecimos el sistema con **guardrails** de entrada, salida y comunicación A2A, **resiliencia** ante agentes caídos y un **allow-list** de agentes permitidos.

- **Stack:** uv + Python 3.13 · LangGraph (A2A nativo, `langgraph dev` en `:2024`) · Gemini vía `langchain-google-genai` (embeddings `models/gemini-embedding-001`) · Redis (redis-stack dockerizado) + `langchain-redis` para el RAG.
- **Repositorio (público):** https://github.com/etmunoz/tarea_1_curso_2
- **Pruebas:** `uv run pytest` -> **70 passed, 1 skipped**.

### 1. Identificación

- **Curso 2 · Sesión 1** — Taller "tu skill es un agente y habla por A2A".
- **Grupo:** NN
- **Integrantes:**
- Fabio Durán
- Gustavo Reyes
- Ezequiel Muñoz
- **Repositorio (público):** https://github.com/etmunoz/tarea_1_curso_2

> Nota sobre el stack: el enunciado del taller (`taller-a2a/`) describe un molde más viejo (un `server.py` en `:8003`, coordinador en `:8002`). Lo tradujimos al stack **real y vigente**: A2A **nativo** de LangGraph, donde los agentes son grafos declarados en `langgraph.json` y publicados por `langgraph dev`. Las ideas del taller (ficha, túnel, coordinador, ronda) se mantienen; cambia la plomería.

### 2. Blueprints (uno por integrante)

El *blueprint* es la ficha de diseño del agente (qué es, qué hace, qué no hace, qué herramientas usa, a quién deriva, dónde se ejecuta).

- **Ezequiel · IA Engineer & Data Scientist** -> `blueprint-ezequiel.md`
- **Fabio** -> `blueprint-fabio.md` (plantilla; la completa Fabio)
- **Gustavo** -> `blueprint-gustavo.md` (plantilla; la completa Gustavo)

En este repositorio el grupo levantó **tres agentes** propios (Ezequiel, Tomás y cocina). Tomás es el segundo agente del integrante Ezequiel, y existe para poder **probar A2A entre los dos computadores** del grupo: en el Mac corre Ezequiel y en el Windows corre Tomás.

### 3. Mi agente (Paso 3)

**Agente de Ezequiel** (`src/agente_ezequiel/`) — Ingeniero de IA y Data Scientist. Recomienda qué técnica de machine learning usar, qué métricas mirar y qué modelo elegir según el tipo de dato. Es un agente `create_agent` (LangGraph) con su **prompt separado** del grafo y sus herramientas en `herramientas.py`:

- `recomendar_tecnica(problema)` — mapea un problema (p. ej. desbalance de clases) a una técnica y el porqué.
- `metricas_recomendadas(tipo_problema)` — qué métricas priorizar.
- `elegir_modelo(tipo_de_dato)` — modelo de arranque según el dato.

- `consultar_modelos(...)` — consulta un **catálogo de modelos en SQLite** (`bd_modelos.py`), de modo que el agente tiene conocimiento "conectado" a una base, no solo texto.

El grafo queda registrado en `langgraph.json` como `agente_ezequiel`. **Qué cambiamos tras verlo funcionar:** separamos el prompt del código, y cuando una herramienta recibe algo fuera de su catálogo responde en una línea con lo que sí cubre (nunca lanza error).

Los otros dos agentes del grupo:

- **Agente de cocina chilena** (`src/agente_cocina/`): 20 recetas chilenas con **costeo** desde SQLite. Herramientas: `listar_recetas`, `ver_receta`, `costear_receta` (costo por ingrediente, total y por persona, en CLP).

- **Agente de Tomás** (`src/agente_tomas/`): Maestro Reiki con **RAG sobre Redis**. Herramientas: `principios_reiki`, `posiciones_manos`, `info_simbolo` y `buscar_reiki` (búsqueda semántica sobre `data/reiki/*` indexado con embeddings de Gemini).

- **Prompt usado** para crearlo: `prompts_ejecutados/04-crear-agente-ezequiel.md`.

## 4. Probarlo solo (Paso 4)

Antes de sumarlo a la red, el agente se prueba aislado: responde a una pregunta **de su tema** usando su herramienta y **rechaza** una de otro tema (no alucina fuera de dominio).

- Comandos y salida: `respuestas/04-crear-agente-ezequiel.md`; notebook paso a paso: `notebooks/ezequiel_paso_a_paso.ipynb`.
- **Evidencia:** `evidencias/01-saltos.png` — ronda real por el coordinador mostrando los "saltos" (pregunta -> ruta del supervisor -> respuesta `[Agente de ...]`), incluido el rechazo (`sin_agente`) de una pregunta fuera de dominio.

## 5. La ficha / Agent Card (Paso 5)

La **Agent Card** es cómo un agente se presenta al mundo A2A: un JSON en `/.well-known/agent-card.json` con `name`, `description` (qué sé / qué no hago) y `skills`. El supervisor **decide a quién enrutar leyendo estas fichas**, así que una buena descripción es clave.

Mecánica concreta en A2A nativo de LangGraph:

1. Cada grafo tiene un `assistant_id` (UUID) que se obtiene con `POST /assistants/search {"graph_id": "<grafo>"}`.
2. La ficha se **aplica** con `PATCH /assistants/<id>` poniendo `name` + `description` desde `src/fichas.py`. Lo hace `tools/configurar_fichas.py`.
3. Se **lee** en `GET /.well-known/agent-card.json?assistant_id=<id>`.

> Detalle operativo importante: `langgraph dev` **recarga en caliente** al cambiar archivos, y esa recarga **borra las fichas aplicadas**. Por eso, tras cualquier cambio hay que volver a correr `configurar_fichas.py`.

- **Evidencia:** `evidencias/02-ficha-agente_ezequiel.json` (y también `02-ficha-agente_tomas.json`, `02-ficha-agente_cocina.json`, `02-ficha-agente_viajes.json`).

## 6. Salir a internet · Cloudflare (Paso 6)

Para que el agente de un computador sea alcanzable desde **otra red**, lo exponemos con un **túnel rápido de Cloudflare**:

```
cloudflared tunnel --url http://127.0.0.1:2024
```

Esto publica el `langgraph dev` local en una URL `https://<algo>.trycloudflare.com`, **solo HTTP**, sin abrir puertos ni tocar el router. La URL es **efímera** (cambia en cada corrida), así que los equipos se intercambian su URL al momento de la ronda.

Decisión de diseño del grupo: **todo** el tráfico entre máquinas va por túneles HTTP de Cloudflare. No usamos la LAN ni nada por TCP: pensamos el escenario como "dos personas en redes distintas". En particular, Tomás **no expone Redis**; expone únicamente su agente (que por dentro usa su RAG). La **misma repo** sirve para ambos equipos: cambia solo la configuración por variables de entorno.

- **Evidencia:** `evidencias/03-tunel.png` — la ficha del agente servida por la URL pública de Cloudflare (alcanzable desde internet).

## 7. Sumar a los compañeros · coordinador (Paso 7)

El **coordinador** (`src/coordinador/`) es un **supervisor** (patrón StateGraph, no handoff): no responde él, **decide** a qué agente mandar la pregunta y reenvía por A2A.

Piezas:

- `catalogo.py` — arma el **catálogo** desde variables de entorno: `A2A_GRAFOS_LOCALES` (agentes del propio `langgraph dev`) y `URLS_AGENTES` (agentes de compañeros, cada uno por su túnel). Cada entrada marca si es **local** o **remoto** y resuelve su ficha (con caché por proceso y TTL, para no pedirla por el túnel en cada paso).

- `graph.py` — el grafo supervisor. El nodo `enrutar` usa `modelo.with_structured_output(Decision)` para elegir **exactamente un nombre del catálogo o "ninguno"**, guiado por las descripciones de las fichas. Luego `consultar` reenvía por A2A y **etiqueta** la respuesta como `[Nombre] ...` (es un dato, no una instrucción). Si nadie cubre la pregunta, `sin_agente` lo dice con cortesía.
- `a2a.py` — cliente A2A genérico: `assistant_id`, `agent_card` y `enviar_mensaje` (`message/send` con `contextId` = UUID = hilo, para que el remoto recuerde su conversación).

Como es **bidireccional**: en el Mac el coordinador tiene locales a Ezequiel y cocina, y remoto a Tomás (Windows); en el Windows es al revés (local Tomás, remotos Ezequiel y cocina). El usuario no nota la diferencia entre local y remoto (**A2A transparente**).

- **Evidencia:** `evidencias/04-catalogo-mac.txt` y `04-catalogo-windows.txt` — el catálogo real de cada equipo (dos locales + un remoto), probado Mac <-> Windows por Cloudflare, en redes distintas.
- Agentes ajenos a sumar por los compañeros: ver `companeros.md`.

## 8. La ronda de preguntas (Paso 8)

Ronda real registrada en `evidencias/05-ronda-mac.md` y `05-ronda-windows.md`. Desde el **Mac** (Ezequiel y cocina locales; Tomás remoto por Cloudflare):

| Pregunta | A quién enrutó el supervisor | ¿Estuvo bien? |
|---|---|---|
| ¿Qué técnica uso si tengo desbalance de clases? | [Agente de Ezequiel] (local) | sí |
| ¿Qué símbolos se aprenden en el nivel 2 de Reiki? | [Agente de Tomás] (**remoto**, por Cloudflare) | sí |
| ¿Cuánto cuesta un pastel de choclo para 4 personas? | [Agente de cocina chilena] (local) | sí |
| ¿Quién ganó el mundial de 1962? | sin_agente (fuera de dominio) | sí (lo rechaza bien) |

Desde el **Windows** la ronda es el **espejo** (Tomás local; Ezequiel y cocina remotos): una pregunta de Reiki la responde Tomás local, y una de métricas de regresión o de costear una cazuela se enruta a los agentes del Mac por Cloudflare.

- **Comentario:** el supervisor eligió correctamente por ficha; derivó al agente remoto (Tomás) por Cloudflare **sin que el usuario note la diferencia**, y rechazó con cortesía lo que ningún agente cubre. Con esto queda demostrada la operatividad A2A en **los dos sentidos**.
- **Evidencia:** `evidencias/05-ronda-mac.md`, `evidencias/05-ronda-windows.md`.

## 9. Cómo correr el proyecto

Requisitos: `uv`, Python 3.13, Docker (para el Redis del RAG) y una `GOOGLE_API_KEY` de Gemini.

```
uv sync                         # instala dependencias (entorno reproducible)
cp .env.example .env            # pega tu GOOGLE_API_KEY en el .env (NUNCA se versiona)
docker compose up -d            # levanta Redis para el RAG de Tomás
uv run langgraph dev            # publica los grafos en http://127.0.0.1:2024
uv run python tools/configurar_fichas.py   # aplica las fichas (Agent Cards)
uv run pytest                   # pruebas: 70 passed, 1 skipped
```

Para la prueba entre dos computadores (plug and play por equipo):

- **Mac:** `scripts/mac_coordinador.sh "<túnel-windows>"` · `scripts/mac_app.sh` · `scripts/mac_tunel.sh`.
- **Windows:** `scripts/win_coordinador.ps1 "<túnel-mac>"` · `scripts/win_app.ps1` · `scripts/win_tunel.ps1`.

Hay además una **app web** mínima (stdlib, `src/webapp/`) para probar agentes uno a uno o vía coordinador, y notebooks paso a paso en `notebooks/`. Guía completa: `taller-a2a/`.

## 10. Robustez (resiliencia + guardrails + allow-list)

Sobre el coordinador implementamos controles de robustez (prompts 44-47), cada uno con pruebas y evidencia.

**(a) Agente remoto no disponible — resiliencia.** `src/coordinador/a2a.py` envuelve toda petición HTTP y convierte conexión caída / timeout / HTTP inválido / JSON ilegible en una excepción tipada `AgenteNoDisponible` (nunca deja escapar un error crudo). El catálogo se **arma igual** marcando al agente caído como "(no disponible ahora)", y el coordinador devuelve un **estado controlado**: «El Agente de X no está disponible temporalmente (motivo). Puedes intentar nuevamente más tarde. Los demás agentes continúan operativos.» Se probó **con el Windows realmente caído**.

**(b) Guardrails en tres puntos** (`src/coordinador/guardrails.py`):

- **Entrada** (`guardia_entrada`): rechaza preguntas vacías, excesivamente largas o con patrones de **inyección/jailbreak** (ES/EN), antes de gastar el LLM.

- **Salida** (`guardia_salida`, **nodo común a todos los caminos**): redacta credenciales (claves de API, tokens, contraseñas, tarjetas), aplica un tope de longitud y evita respuestas vacías. Que sea un nodo final garantiza que **ninguna** respuesta salga sin pasar por el control.
- **Comunicación A2A** (`sanear_respuesta_remota`): la respuesta de un agente remoto se trata como **dato no confiable**; si trae una instrucción incrustada, se marca "se muestra como dato y NO se obedece".

**(c) Allow-list de agentes** (`A2A_AGENTES_PERMITIDOS`): lista blanca explícita de los grafos que el coordinador puede invocar (sus "tools"). Aunque `URLS_AGENTES` traiga un túnel colado o mal configurado, **no se ejecuta** si no está aprobado. Si la variable no se define, se permiten todos (compatibilidad hacia atrás).

El flujo del grafo resultante (ver `evidencias/07-grafo-coordinador.svg` y `09-diagramas.md`):

```
__start__  ->  guardia_entrada  ->  enrutar  ->  {consultar | sin_agente}  ->  guardia_salida  ->  __end__
                   ■■■(bloqueado)■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

- **Pruebas:** `tests/test_resiliencia_a2a.py`, `tests/test_guardrails.py`, `tests/test_salida_y_allowlist.py`.
- **Evidencia:** `evidencias/06-robustez-mac.md` (resiliencia con túnel caído, batería de ataques de entrada detenidos, redacción de salida y allow-list excluyendo un intruso).

## 11. Arquitectura y diagramas

- `evidencias/07-grafo-coordinador.svg` — el grafo del coordinador (con los guardrails).
- `evidencias/08-arquitectura.svg` — la arquitectura completa: Mac (Ezequiel, cocina, viajes) y Windows (Tomás + RAG en Redis), cada uno con su `langgraph dev`, coordinador y app web, unidos por túneles HTTP de Cloudflare (A2A bidireccional).
- `evidencias/09-diagramas.md` — índice con el **mermaid auténtico** del grafo (extraído del grafo compilado) + los dos SVG.

## 12. Evidencias incluidas

Lista y cómo producir cada una: `evidencias/README.md`.

- `01-saltos.png`, `03-tunel.png` — generadas desde la corrida real (coordinador y túnel).
- `02-ficha-agente_*.json` — Agent Cards aplicadas.
- `04-catalogo-mac.txt`, `04-catalogo-windows.txt` — catálogo del coordinador en cada equipo.
- `05-ronda-mac.md`, `05-ronda-windows.md` — ronda de preguntas en cada equipo (A2A bidireccional).
- `06-robustez-mac.md` — resiliencia + guardrails + allow-list (prompts 44-47).
- `07-grafo-coordinador.svg`, `08-arquitectura.svg`, `09-diagramas.md` — diagramas.

## 13. Convenciones y trazabilidad del grupo

- **Encabezado de grupo** en todo archivo que admite comentarios (verificado con `tools/aplicar_encabezados.py --check`).
- **Cada prompt** queda registrado en `prompts_ejecutados/NN-slug.md` y **cada respuesta** (con la salida real, lo que funciona y lo que falla) en `respuestas/NN-slug.md`, indexados en `prompts_ejecutados/README.md`. A la fecha de esta entrega hay **47 prompts** registrados.
- **Seguridad:** el `.env` real (con la clave) **nunca** se versiona (`.gitignore`); la `GOOGLE_API_KEY` no aparece en el repositorio.